Analyzing the Impact of Domain Features on Heuristic Search Performance in Grid-Based Planning

André Vieira Ribeiro
andre.vieiraribeiro@stud.unibas.ch
Department of Mathematics and Computer Science
University of Basel
Basel, Switzerland

***Abstract*—Classical planning research commonly evaluates planning algorithms using benchmark domains that differ substantially in structure, mechanics and modeling style. As a result, it is often difficult to determine whether differences in planner performance originate from the planning algorithm itself or from the characteristics of the domain being solved. This work investigates how specific domain mechanics influence heuristic planning performance within a single consistent environment.**

To achieve this, a configurable grid-based planning domain inspired by the puzzle game *Dungeons of Dreadrock* is implemented in PDDL and solved using the Fast Downward planning system. The domain includes mechanics such as walls, keys, doors, portals, zombies, pressure plates and fireballs, which can be activated individually or combined across different problem instances. The additive, FF and LM-cut heuristics are evaluated using metrics including runtime, expanded states, generated states and plan length over multiple grid sizes and difficulty levels.

The experimental results show that mechanics involving conditional effects, dynamic hazards and dependency relations significantly increase search complexity and influence heuristic behavior differently. Furthermore, increasing grid size and mechanic interaction leads to substantial growth in search effort. The findings demonstrate the importance of controlled domain design when analyzing heuristic planning performance.

I. Introduction

Since the dawn of Artificial Intelligence, one of its fundamental ambitions has been to design general-purpose algorithms capable of finding a sequence of actions that lead from an initial state to a desired goal. This problem is known as classical planning and has been widely studied for decades [1].

Modern planning systems are traditionally evaluated by measuring their performance across benchmark domains provided by the International Planning Competition (IPC) [2], [3]. These benchmark domains are drawn from varied application areas, from logistics and robotics to puzzle environments and can range from simple organizational tasks to more elaborate puzzle games that combine simple rules with high computational complexity. However, these domains differ significantly in structure and modeling assumptions, which can influence the performance of the planner beyond algorithmic improvements.

Prior research on heuristic planning [4], has shown that the heuristic performance of a planning algorithm, such as the Fast Downward planning system [5] and the FF planning system [6] are heavily dependent on the structure of the benchmark domain.

This makes it very difficult to interpret comparisons between different domains. For example, a planner that does well in a transportation domain might be taking advantage of heuristics that fit resource flow really well, while another planner might shine in spatial or deadlock-heavy puzzles like Sokoban, which is known for its unique structural challenges [7]. Since these mechanics show up in different environments and representations, it's hard to tell if the performance gaps are due to the planner's own abilities or just the specific characteristics of the domain it's working in.

To address this issue, this work creates a single planning domain in which multiple mechanics can be activated independently or in combination within a single unified model. This allows us to analyze how specific structural features influence heuristic search performance while keeping the underlying domain representation consistent.

The rest of this paper is structured as follows: First, the methodology describes the design approach for the new planning domain and experimental setup, followed by results and analysis. Finally, we present related works and conclusions.

II. Related works

A. Benchmark Domains and the International Planning Competition

Classical planning research relies heavily on benchmark domains to evaluate and compare planning systems under controlled conditions. Early planning approaches such as STRIPS introduced formal representations of actions through preconditions and effects, establishing the foundations of automated planning research [8]. Later work expanded these ideas into more general planning frameworks and domain-independent planning systems [1].

A major milestone for benchmark standardization was the introduction of the Planning Domain Definition Language (PDDL), which provided a common language for representing planning domains and problem instances [9]. The introduction

of PDDL enabled planners to be evaluated on shared benchmarks and became closely connected to the International Planning Competition (IPC), first organized in 1998 [3]. Since then, the IPC has played an important role in advancing planning research by providing standardized evaluation environments and encouraging the development of increasingly scalable planning systems [2].

Benchmark domains are especially important because the effectiveness of heuristic search strongly depends on domain structure. Different domains emphasize different reasoning challenges, such as spatial navigation, resource dependencies, deadlocks or long-term goal interactions. Helmert showed that many standard benchmark domains already contain computationally difficult planning structures despite relatively simple rules [10]. This motivates the design of benchmark environments that isolate different sources of search complexity in a controlled and reproducible way.

B. Puzzle and Game-Inspired Planning Domains

Among the most influential benchmark environments in planning research are grid-based puzzle domains, particularly Sokoban. In Sokoban, the player must push boxes to goal locations while navigating confined spaces and avoiding irreversible mistakes. Although the rules are simple, the domain creates highly challenging search problems and has been shown to be PSPACE-complete [7].

Sokoban became especially relevant for heuristic planning because it naturally combines spatial reasoning, dependency constraints and deadlock situations. Research by Junghanns and Schaeffer demonstrated that deadlock detection is crucial for efficient search performance in such environments [11]. Related puzzle environments such as PushPush further illustrated the computational difficulty of movement-based puzzle domains [12]. These works established puzzle games as valuable benchmark environments for studying search behavior and planning difficulty.

Puzzle and game-inspired domains are particularly useful because they provide structured environments with clearly defined rules while still producing complex search spaces [7], [12]. Many mechanics commonly found in puzzle games, such as navigation constraints, environmental hazards and progression dependencies, can be represented naturally within classical planning frameworks [1]. This makes such domains well suited for evaluating how different planning systems and heuristics react to varying problem structures [4], [6].

Game-inspired environments have also gained importance in broader AI research. General Video Game AI research investigates how intelligent agents can operate across diverse rule systems and environments [13]. Although these approaches often focus on real-time decision making rather than classical planning, they similarly demonstrate that games provide scalable and structured environments for evaluating intelligent behavior.

C. Heuristic Search in Classical Planning

Heuristic search is one of the most successful approaches in domain-independent planning. Instead of exploring the search space blindly, heuristic planners estimate the distance from the current state to the goal and prioritize states that appear more promising [4]. Modern planning systems such as Fast Downward provide flexible architectures for combining different search strategies and heuristics within the same framework [5], [14].

Many widely used heuristics are based on relaxed planning problems, where certain constraints of the original task are ignored to simplify computation. The FF heuristic constructs relaxed plans to estimate goal distance and became highly influential due to its practical efficiency [6]. Other approaches focus more strongly on structural properties of planning tasks. Landmark-based heuristics identify facts that must necessarily be achieved in every valid solution and use them to guide search [15], [16]. Similarly, causal graph heuristics analyze dependencies between variables to estimate planning difficulty [17].

Previous research has shown that heuristic performance can vary significantly depending on the structure of the planning domain. Some heuristics perform well in navigation-heavy problems, while others are better suited for handling dependencies or dead-end states. This motivates the evaluation strategy of this work, where multiple heuristics are compared across different combinations of mechanics and problem structures in order to analyze how domain features influence heuristic-guided search behavior.

III. Methodology

A. Research Questions and Experimental Strategy

After introducing the motivation for this study, this section gives an overview of the three main research questions that guide this research, in order to investigate how domain mechanics influence heuristic planning performance in a controlled and reproducible manner.

- **Question 1**: Does the inclusion of different domain features (e.g., resource constraints, dependency relations, conditional interactions) affect the performance of heuristic-guided search?
- **Question 2**: Do the additive, FF, and LM-cut heuristics differ in their performance across varying combinations of domain features?
- **Question 3**: Does increasing problem difficulty and grid size influence search performance under different structural feature configurations?

To answer these questions, we designed and implemented a planning domain using PDDL, which is inspired by the grid-based puzzle game "Dungeons of Dreadrock" [18]. From this game, we select a subset of distinct mechanics that can be used independently. This allows us to create multiple problem instances (game levels) that differ in mechanics, structure and complexity, while keeping the modeling context consistent. We use the Fast Downward planning algorithm [5] to solve the instances, with three different heuristics: the additive heuristic [4], the FF heuristic [6] and the LM-cut heuristic. The planners performance is then evaluated using standard metrics like runtime and search effort over problems that vary in size and difficulty. This methodology allows us to properly analyze how

individual mechanics and problem scale affect the heuristic search behavior, all within a single and consistent domain.

B. Inspiration from Grid-Based Puzzle Games

This work draws inspiration from *Dungeons of Dreadrock*, a tile-based puzzle game, developed and published by Prof. Dr. Christoph Minnameier in 2022 [18]. The game requires the player to travel through dungeon levels and reach an exit while interacting with mechanics such as zombies, projectiles, keys and doors. Unlike traditional puzzle benchmarks such as Sokoban, the game combines multiple mechanics within the same environment, introducing different forms of reasoning including dependency handling, conditional interactions and spatial constraints.

Rather than reproducing the entire game, this work selects a small subset of mechanics that can be represented within classical planning. Mechanics involving randomness or real-time behavior are excluded in order to preserve determinism and compatibility with domain-independent planners.

C. PDDL Modeling Concepts Used

The domain is encoded in the Planning Domain Definition language (PDDL) [9] to ensure compatibility with domain-independent planners. A PDDL planning model consists of two complementary files: a *domain file* and a *problem file*.

The domain file defines the general rules of the environment through predicates and actions.

```
(define (domain D-name)
   (:requirements ...)
   (:predicates ...)
   (:action move
      :parameters (...)
      :precondition (...)
      :effect (...))
)
```

Fig. 1: Simplified structure of the PDDL domain file used in this work.

The problem file specifies a concrete instance by declaring objects, the initial state and the goal condition. This separation allows us to easily create multiple problem instances that share the same underlying domain structure.

```
(define (problem P-name)
   (:domain D-name)

   (:objects ...)
   (:init ...)
   (:goal ...)
)
```

Fig. 2: Simplified structure of the PDDL problem file used for individual level instances.

The domain uses features from PDDL 2.1 [19], including *negative preconditions*, *quantifiers* (`forall`) and *conditional effects*. These features are required to model interactive mechanics and context-dependent state transitions within the planning environment.

D. Domain Mechanics

The planning domain is constructed from a small set of mechanics where each represents a distinct source of computational complexity in heuristic search. The design focuses on isolating structural features that influence planning performance in different ways, such as spatial constraints, resource dependencies, dynamic hazards, conditional effects, and non-local transitions.

Each mechanic is represented both conceptually and through a visual icon, which is used consistently across all figures of problem instances in later sections.

Mechanic	Description
Player	Player character that moves across the grid and interacts with the environment. (Core entity)
Goal	Target tile that must be reached by the player to complete the level. (Goal condition)
Walls	Block tiles and build restrictions within the environment, limiting possible movement paths. (Spatial constraint)
Zombies	Enemies that move back and forth along a predefined path. The player dies upon contact unless equipped with a sword. Zombies move one step every round. (Dynamic hazard)
Fireballs	Hazards that travel in a straight line along a predefined path until hitting a wall, after which they disappear. The player dies upon contact. (Dynamic hazard)
Pressure Plate	Activates a fireball and causes it to traverse its predefined path. (Conditional effect)
Portals	Allow the player to instantly travel between two distant tiles on the grid. Travel is possible in both directions. (Non-local transition)
Sword	Collectible item that allows the player to eliminate zombies and survive enemy encounters. (Resource interaction)
Key	Collectible item that must be inserted into a matching keyhole to open a door. (Dependency mechanic)
Keyhole	Interactive object that requires the corresponding key to unlock an associated door. (Dependency mechanic)
Door	Blocks player movement until opened by inserting the correct key into its corresponding keyhole. (Dependency mechanic)

TABLE I: Overview of mechanics used in the planning domain. Source of icons: https://www.flaticon.com

All mechanics are defined within a shared grid-based environment, where their interaction is controlled to allow isolated or combined activation depending on the experimental con-

figuration. This design enables systematic comparison of how each structural feature influences heuristic search performance under identical planning conditions.

E. Base Domain Structure

This section describes the underlying structure of the *domain file* that enables the implementation of the mechanics introduced in the previous section.

The planning domain is modeled as a grid-based environment where the player moves between connected tiles to reach a goal location. Spatial relationships are represented through the predicate `adjacent(tile_a, tile_b)` allowing the planner to reason about movement independently from the map specific layout and. Walls and closed doors are used to model temporarily blocked tiles that restrict traversal.

The domain maintains the current world state through predicates describing the player position, resources and active hazards. Interactive objects such as keys, portals, zombies and fireballs are represented through predicates encoding their location and state. Dynamic hazards are updated automatically after each player action using conditional effects and quantified updates available in PDDL 2.1 [19]. This allows zombies and fireballs to continuously traverse predefined paths during execution. Table II summarizes the main actions implemented in the domain.

Action	Purpose
`move`	Moves the player between adjacent tiles while updating dynamic hazards.
`collect-key`	Allows the player to collect a key.
`insert-key`	Unlocks a door using a matching key and keyhole.
`collect-sword`	Allows the player to obtain a sword.
`attack-zombie`	Eliminates a zombie using the sword.
`teleport`	Moves the player through portals to distant tiles.
`activate-fireball`	Activates a fireball through a pressure plate.
`stay-still`	Advances hazard movement while the player waits.
`die-by-fire`	Represents player death caused by a fireball.
`die-by-zombie`	Represents player death caused by a zombie.

TABLE II: Main actions implemented in the planning domain.

This modular structure allows individual mechanics to be isolated or combined across problem instances without modifying the underlying domain definition. As a result, different mechanics can be evaluated systematically under consistent planning conditions.

F. Configurable Problem Instances

The experimental evaluation is performed on configurable problem instances that share the same domain definition while varying in mechanics, layout and difficulty. Each instance is represented through a separate PDDL problem file defining the initial world state and goal configuration.

Each problem file specifies the placement and behavior of all mechanics used in the level, including hazards, resources and interactive objects. The player starts at a predefined tile and the problem is solved once the goal tile is reached while remaining alive.

To analyze the influence of individual mechanics, most instances isolate a single mechanic or mechanic category whenever possible. Walls are included in many levels to preserve spatial structure, while mechanics such as portals, zombies, fireballs and key-door dependencies are evaluated independently or in selected combinations. This enables controlled comparison of how different mechanics influence heuristic search performance.

Difficulty is increased through larger grid sizes, more complex layouts and additional interaction dependencies, as illustrated in Figure 3. In addition, all levels across all difficulty settings are provided in the Appendix.

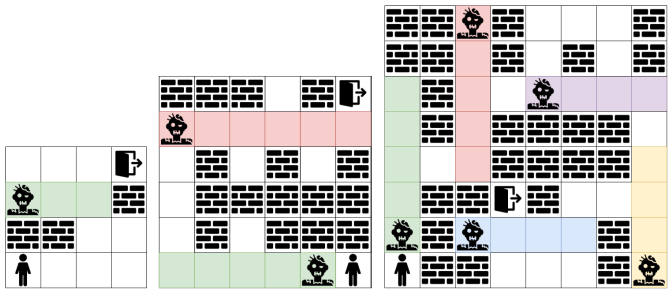
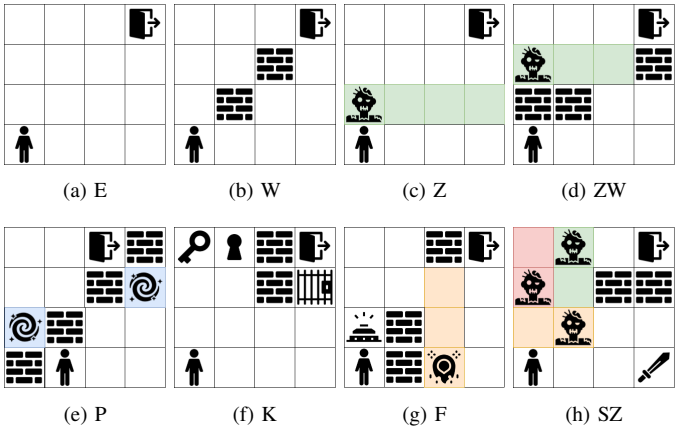


Fig. 3: Grid size categories used for the generated problem instances. The easy, medium and hard configurations use grid sizes of 4x4, 6x6 and 8x8 respectively. Difficulty is increased through larger environments, more constrained navigation paths and additional interaction dependencies.

The easy-difficulty instances shown in Figure 4 illustrate the individual mechanics and mechanic combinations evaluated throughout the experiments.



(a) E (b) W (c) Z (d) ZW

(e) P (f) K (g) F (h) SZ

Fig. 4: Easy-difficulty problem instances used throughout the experiments. Each level isolates a specific mechanic or mechanic combination: Empty (E), Walls (W), Portal (P), Zombie (Z), Key-Door (K), Zombie+Walls (ZW), Fireball+Pressure Plate (F), and Sword+Zombie (SZ).

G. Heuristic Strategies

All experiments are conducted using the Fast Downward planning system [5], executed with three different heuristic configurations: the additive heuristic, the FF heuristic, and the LM-cut heuristic [4], [6], [16]. These heuristics were chosen because they have different approaches on how to estimate goal distance and are widely used in modern planners such as Fast Downward.

The search strategy used is *Eager Greedy Best-First Search*, which is a heuristic search algorithm that always prioritizes states with the lowest heuristic value. This ensures that all problem instances are solved by the same underlying search algorithm, while only the heuristic guidance is varied.

Heuristic	Description
Additive	Estimates goal distance by decomposing the problem into subgoals and summing individual costs. Fast but may ignore interactions between subgoals [4].
FF	Uses relaxed planning by ignoring delete effects. The heuristic value is based on the length of a relaxed solution. Strong practical performance [6].
LM-cut	Computes a lower bound by extracting landmark-based cost partitions, resulting in an admissible heuristic. Typically more informed but more expensive [16].

TABLE III: Heuristic functions used in the evaluation and their properties.

Since these heuristics differ in their computational cost, accuracy and properties, we can observe how different heuristic strategies interact with the same domain structure under the same search conditions.

H. Evaluation Metrics

To evaluate how the introduced mechanics affect the performance and difficulty of the planning system, we select only a small set of metrics that reflect both *computational effort* and *plan complexity*. The following metrics are used:

Metric	Description
Search Time	Time required by the planner to find a solution, reflecting overall computational effort.
Expanded States	Number of states expanded during search, indicating search space exploration effort.
Generated States	Total number of successor states generated during search, reflecting branching behavior.
Plan Length	Number of actions in the final solution, measuring solution complexity independently of search effort.

TABLE IV: Evaluation metrics used to assess planning performance and solution complexity.

These metrics together allow us to distinguish between problems that are hard because the solution is long, and problems that are hard because the search space is large.

I. Experimental Setup

The planner is run on a uniform hardware setup (Intel Core i5 8th Gen CPU, 16 GB RAM) with a time limit of 300 seconds per instance and unit-cost actions. All experiments use the same domain file and a systematically generated set of problem instances varying in active mechanics, grid size, and difficulty level.

The full implementation, including the PDDL domain and generated problem instances, is available in the accompanying GitHub repository: https://github.com/AndreVRibeiro/pddl-mechanic-analysis

IV. Results

Table V gives an overview of the abbreviations used to define the problem instances in the experiments. The evaluation is divided into three parts with the goal of answering the three research questions established previously: (1) the impact of individual mechanics, (2) a comparison of heuristic strategies, and (3) the effect of increasing difficulty and grid size.

Abbrev.	Mechanic	Type
E	Empty	baseline
W	Walls	spatial constraint
Z	Zombies	dynamic hazard
ZW	Zombies + Walls	spatial + hazard
P	Portals + Walls	non-local transition
K	Keys + Doors + Walls	dependency
F	Fireballs + Plates + Walls	conditional effects
SZ	Sword + Zombies + Walls	interaction + mitigation

TABLE V: Abbreviations used for mechanics in all result figures.

A. Impact of Domain Mechanics on Search Performance

The first experiment investigates how different mechanics affect planning difficulty, using the FF heuristic on medium-sized problem instances. Figures 5, 6, and 7 show the generated states, expanded states, search time, and resulting plan length for each mechanic configuration.

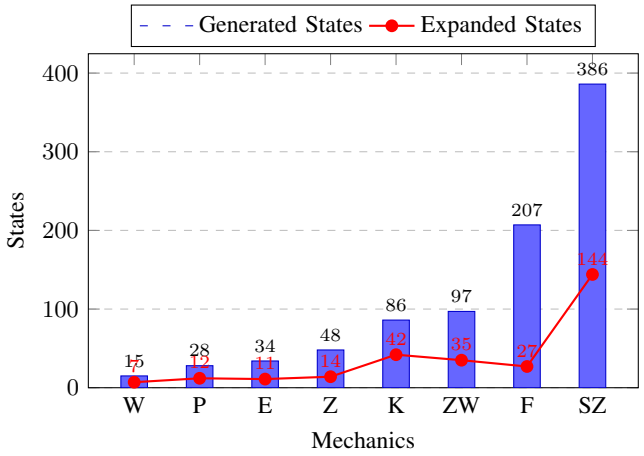


Fig. 5: Comparison of generated and expanded states across medium-difficulty mechanics using the FF heuristic.

Figure 5 shows the number of expanded and generated states for each mechanic configuration. Simpler environments

such as the empty grid and wall-only levels produce relatively small search spaces. In contrast, mechanics involving enemies and conditional interactions significantly increase the number of explored states. The largest increase can be observed in the *Sword + Zombies* configuration, where the planner must reason not only about movement but also about combat interactions and survival constraints. This suggests that combining multiple mechanics creates substantially more complex search spaces than isolated mechanics alone.

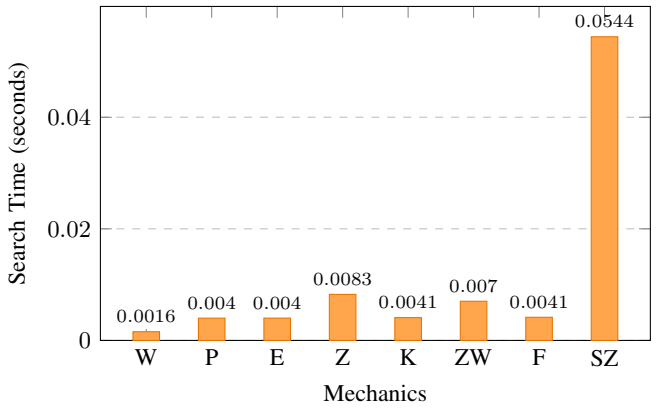


Fig. 6: Search time across medium-difficulty mechanics using the FF heuristic.

Figure 6 presents the corresponding search times. Most mechanics remain within a very small time range between approximately 0.001 and 0.008 seconds, despite differences in search-space size. This indicates that the tested instances are still relatively small and can be solved efficiently by the planner. However, the Sword + Zombies configuration stands out clearly with a search time of approximately 0.054 seconds, which is substantially larger than all other mechanics. This reflects the strong increase in search complexity caused by combining combat interactions, dynamic hazards, and navigation constraints within the same problem instance.

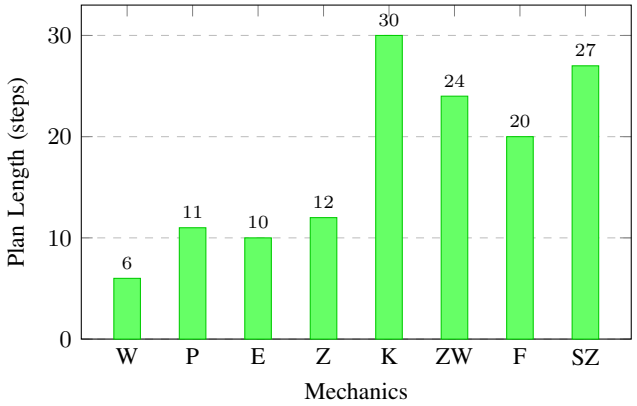


Fig. 7: Plan length across medium-difficulty mechanics using the FF heuristic.

Finally, Figure 7 compares the resulting plan lengths. Key and door mechanics produce longer solution plans because actions must follow a strict dependency order. However, long plans do not always imply high search difficulty, as some shorter zombie-based levels still generate substantially larger search spaces. This demonstrates that structural interactions between mechanics have a stronger impact on heuristic search performance than solution length alone.

B. Comparison of Heuristic Strategies

The second experiment compares the FF, additive and LM-cut heuristics across the generated medium sized problem instances. Figure 8 presents the average number of generated and expanded states for each heuristic, while Table VI provides the exact values per mechanic.

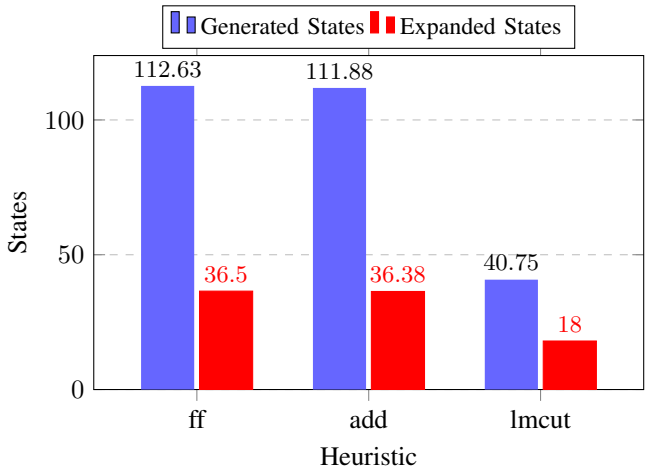


Fig. 8: Mean performance comparison of FF, additive, and LM-cut heuristics across all mechanics and problem instances.

The FF and additive heuristics behave almost identically across all 8 levels. Their generated and expanded state counts differ only slightly, which suggests that both heuristics provide very similar guidance in this domain.

LM-cut on the other hand, generates and expands far fewer states on average and thus appears to perform far better. However, as shown in table VI, LM-cut does not evaluate levels containing zombies, fireballs and sword + zombies configurations. This is because the LM-cut heuristic does not support conditional statements and as a result 4 levels do not contribute to the reported averages.

Mechanic	**FF**		**ADD**		**LM-cut**	
	Exp.	**Gen.**	**Exp.**	**Gen.**	**Exp.**	**Gen.**
E	11	34	11	34	11	34
W	7	15	7	15	7	15
P	12	28	12	28	12	28
Z	14	48	14	48	–	–
ZW	35	97	33	89	–	–
K	42	86	43	88	42	86
F	27	207	27	207	–	–
SZ	144	386	144	386	–	–

TABLE VI: Comparison of expanded and generated states across mechanics and heuristic configurations for medium-difficulty instances.

This is important because those excluded levels are also the most computationally expensive problem instances the

domain. Consequently, the lower average values of the LM-cut heuristic are not because of better heuristic guidance, but because the hardest levels were omitted from the evaluation. For the other mechanics that are supported by all heuristics, LM-cut performs similarly to FF and additive.

C. Influence of Difficulty and Grid Size

The last experiment analyzes how increasing grid size and level difficulty influence search complexity. Figures 9 and 10 show the scaling behavior of expanded and generated states across easy, medium, and hard configurations.

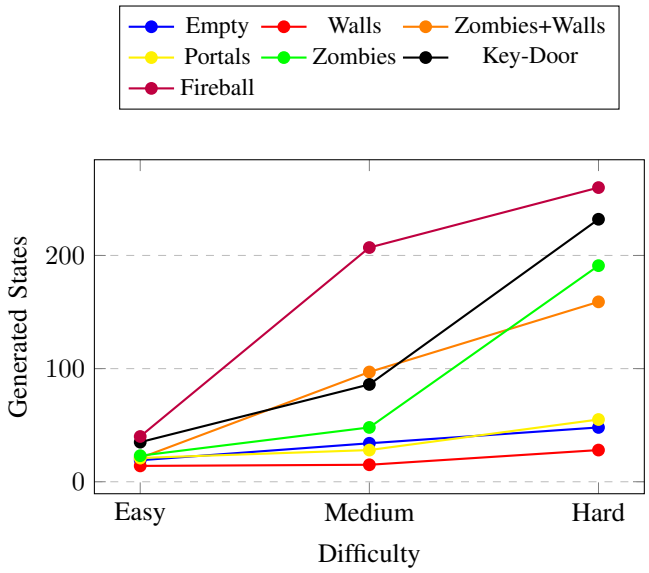


Fig. 9: Scaling of generated states across difficulty levels for different mechanics.

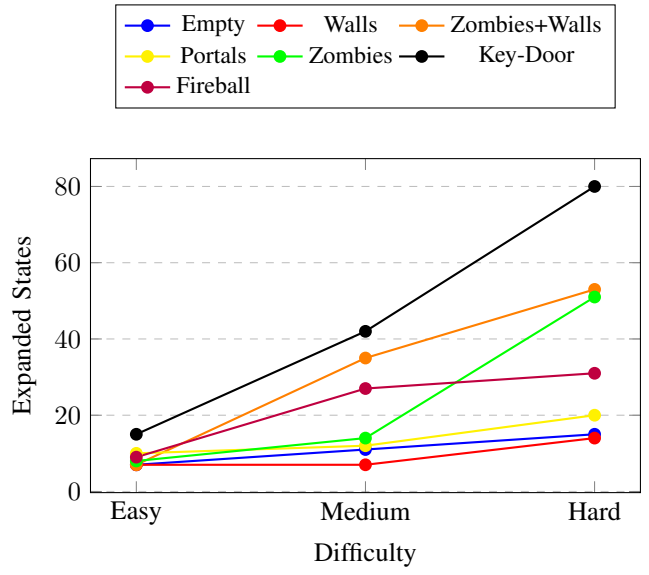


Fig. 10: Scaling of expanded states across difficulty levels for different mechanics.

As would be expected, larger problem instances lead to increased search effort. However, this differs substantially depending on the mechanic type. Empty and wall-based levels scale relatively moderately because the planner mainly performs spatial navigation. In contrast, mechanics involving hazards, dependencies or interactions grow much more rapidly as the environment becomes larger and more complex.

Zombie-based levels show strong growth because moving hazards increase the number of reachable and unsafe states that must be considered during search. Key-Door levels also scale strong due to the additional dependency chains and ordering constraints. Fireball mechanics also exhibit a similar trend because conditional effects and triggered hazards increase branching during exploration.

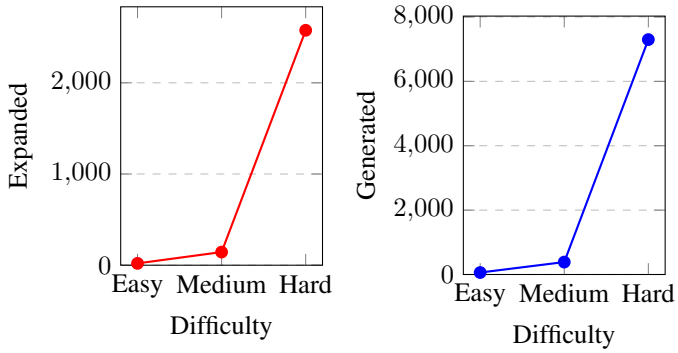


Fig. 11: Scaling behavior of the Sword+Zombies configuration across difficulty levels.

The sword+zombies configuration in Figure 11 demonstrates an extreme increase in search complexity as difficulty increases, specially in generated states. This shows that combining combat interactions with dynamic hazards produces a disproportionately large search space.

V. Conclusion

This paper analyzed the effect of different game-inspired mechanics on the heuristic planning performance in PDDL-based environments. We constructed a configurable grid-based world domain where mechanics like walls, zombies, portals, keys and doors, fireballs, and combat interactions could be systematically enabled or combined. The experiments used a fixed planning domain and multiple varying problem instances, which allowed for a controlled environment for comparing the effects of specific mechanics on search behavior, plan structure and scalability.

The results show that the different mechanics influence planning difficulty in different ways. Search complexity was only slightly impacted by simple spatial constraints such as walls, while mechanics with dynamic hazards or interaction dependencies led to much larger search spaces. In particular, zombie-based environments and conditional mechanics such as fireballs increased the number of generated states and the overall search effort. The most pronounced effect was in the Sword+Zombies setup where the planner needed to reason about movement, survival and combat at the same time. This combination resulted in a large increase in generated and expanded states, especially in larger environments, clearly

showing how mechanics can generate exponential growth in planning complexity.

The experiments also demonstrated that plan length alone is not a reliable indicator of problem difficulty. Some mechanics like keys and doors, led to relatively long plans, as the actions had to be performed in a strict order of dependency. However, other mechanics led to far larger search spaces, even if the solutions were shorter. This suggests that the length of solutions is less important for the performance of heuristic planners and that they are more strongly affected by branching factors, dead-end states and dependencies of interaction.

The comparison between heuristic strategies showed that the FF heuristic, additive heuristic and the LM-cut heuristic produced almost the identical generated and expanded state counts across most evaluated instances. This suggests that, within the tested domain, the overall search behavior was influenced more by the domain structure itself than by differences between heuristic estimators. The main difference was that LM-cut could not evaluate levels with conditional effects and thus excluded several mechanics from the comparison. Consequently, the lower average values obtained for LM-cut do not imply better performance but rather are related to the reduced number of problem instances under evaluation.

Future work could extend this framework in several directions. A first step would be to introduce larger and more diverse grid sizes to better analyze scaling behavior under increased state-space complexity. Additionally, expanding the set of mechanics and exploring more combinations between them could help reveal more subtle interaction effects. Another promising direction would be to incorporate stochastic or probabilistic mechanics, which would more closely resemble real game environments and further challenge heuristic guidance. Finally, evaluating additional planners and heuristic strategies would allow for a more general assessment of how robust different planning approaches are under complex, interactive domain structures.

Overall, this work demonstrates that even relatively simple game-inspired mechanics can have a strong and sometimes non-linear impact on heuristic search behavior, emphasizing the importance of domain design when evaluating planning systems.

REFERENCES

[1] M. Ghallab, D. Nau, and P. Traverso, *Automated Planning: Theory and Practice*. San Francisco, CA, USA: Morgan Kaufmann, 2004.

[2] D. Long and M. Fox, "The 3rd international planning competition: Results and analysis," *Journal of Artificial Intelligence Research*, vol. 20, pp. 1–59, 2003.

[3] D. M. McDermott, "The 1998 ai planning systems competition," *AI magazine*, vol. 21, no. 2, pp. 35–35, 2000.

[4] B. Bonet and H. Geffner, "Planning as heuristic search," *Artificial Intelligence*, vol. 129, no. 1-2, pp. 5–33, 2001.

[5] M. Helmert, "The fast downward planning system," *Journal of Artificial Intelligence Research*, vol. 26, pp. 191–246, 2006.

[6] J. Hoffmann and B. Nebel, "The ff planning system: Fast plan generation through heuristic search," *Journal of Artificial Intelligence Research*, vol. 14, pp. 253–302, 2001.

[7] J. C. Culberson, "Sokoban is pspace-complete," University of Alberta, Edmonton, Canada, Tech. Rep. TR97-02, 1997.

[8] R. E. Fikes and N. J. Nilsson, "Strips: A new approach to the application of theorem proving to problem solving," *Artificial Intelligence*, vol. 2, no. 3–4, pp. 189–208, 1971.

[9] C. Aeronautiques, A. Howe, C. Knoblock, I. D. McDermott, A. Ram, M. Veloso, D. Weld, D. W. Sri, A. Barrett, D. Christianson *et al.*, "Pddl—the planning domain definition language," *Technical Report, Tech. Rep.*, 1998.

[10] M. Helmert, "Complexity results for standard benchmark domains in planning," *Artificial Intelligence*, vol. 143, no. 2, pp. 219–262, 2003.

[11] A. Junghanns and J. Schaeffer, "Single-agent search in the presence of deadlocks," in *AAAI/IAAI*, 1998, pp. 419–425.

[12] E. D. Demaine, M. L. Demaine, and J. O'Rourke, "Pushpush and push-1 are np-hard in 2d," *arXiv preprint cs/0007021*, 2000.

[13] D. Perez-Liebana, S. Samothrakis, J. Togelius, T. Schaul, and S. Lucas, "General video game ai: Competition, challenges and opportunities," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, 2016.

[14] M. Helmert, G. Röger, and E. Karpas, "Fast downward stone soup: A baseline for building planner portfolios," in *ICAPS 2011 Workshop on Planning and Learning*, vol. 2835, no. 8, 2011.

[15] J. Hoffmann, J. Porteous, and L. Sebastia, "Ordered landmarks in planning," *Journal of Artificial Intelligence Research*, vol. 22, pp. 215–278, 2004.

[16] M. Helmert and C. Domshlak, "Landmarks, critical paths and abstractions: what's the difference anyway?" in *Proceedings of the international conference on automated planning and scheduling*, vol. 19, 2009, pp. 162–169.

[17] M. Helmert, "A planning heuristic based on causal graph analysis." in *ICAPS*, vol. 16, 2004, pp. 161–170.

[18] C. P. D. K. W. Minnameier, "Dungeons of dreadrock," https://store.steampowered.com/app/1896880/Dungeons_of_Dreadrock/, 2022, accessed: 2026-04-24.

[19] M. Fox and D. Long, "Pddl2. 1: An extension to pddl for expressing temporal planning domains," *Journal of artificial intelligence research*, vol. 20, pp. 61–124, 2003.

VI. Appendix

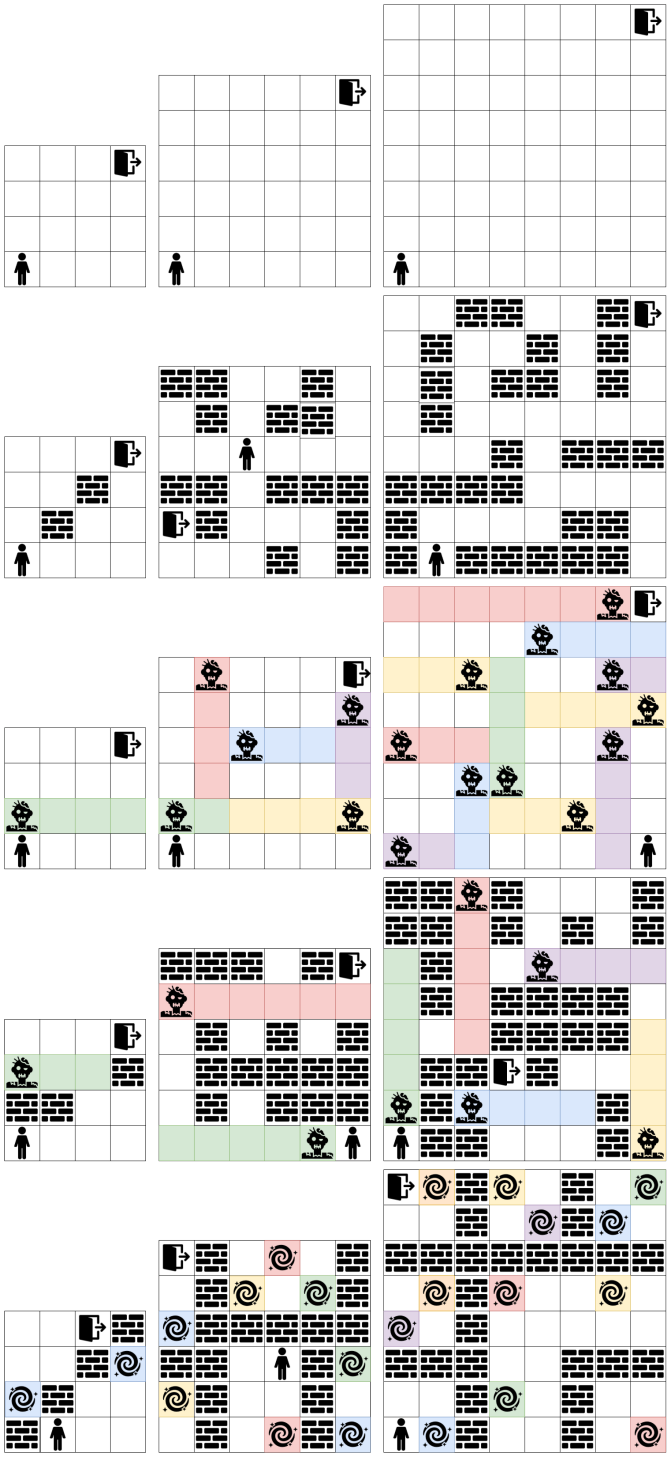


Fig. 12: Overview of all generated problem instances across easy, medium, and hard difficulty levels used in the experimental evaluation. Each level represents a distinct combination of mechanics within a grid-based environment.

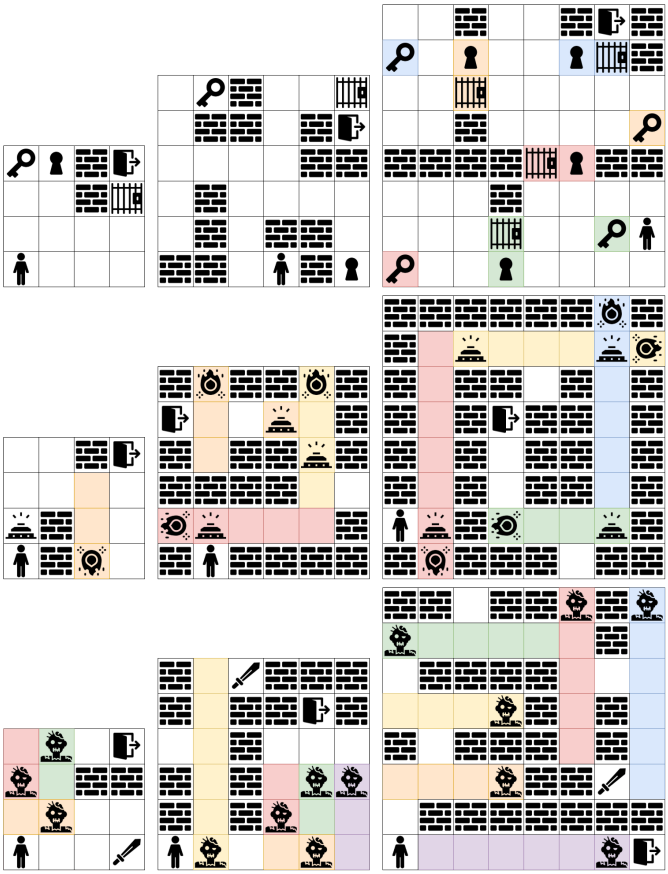


Fig. 13: Overview of all generated problem instances across easy, medium, and hard difficulty levels used in the experimental evaluation. Each level represents a distinct combination of mechanics within a grid-based environment.

VII. Declaration of Independent Authorship

I attest with my signature that I have completed this paper independently and without any assistance from third parties and that the information concerning the sources used in this paper is true and complete in every respect. All sources that have been quoted or paraphrased have been referenced accordingly.

Additionally, I affirm that any text passages written with the help of AI-supported technology are marked as such, including a reference to the AI-supported program used. This paper may be checked for plagiarism and use of AI-supported technology using appropriate software. I understand that unethical conduct may lead to a grade of 1 or “fail” or to expulsion from the course of studies.

I have taken note of the fact that in the event of a justified suspicion of the unauthorized or undisclosed use of AI in written performance assessments, I am upon request obligated to cooperate in confirming or ruling out the suspicion, for example by attending an interview.

Basel, 28 May 2026

André Vieira Ribeiro

VIII. Record of AI Tools used

AI Tool	Type of Use	Affected Parts	Date	Remarks
ChatGPT (OpenAI)	Brainstorming and structuring of report sections and experimental workflow	Overall report structure	May–June 2026	Used to discuss suitable section ordering and scientific presentation structure.
ChatGPT (OpenAI)	Language refinement and phrasing suggestions	Entire paper	May–June 2026	Used to improve grammar, sentence clarity, and academic writing style.
ChatGPT (OpenAI)	Identification of redundant passages and shortening of text	Introduction, methodology, results, conclusion	May–June 2026	Suggestions were manually reviewed and adapted where necessary.
ChatGPT (OpenAI)	Feedback on scientific writing quality and coherence	Entire paper	May–June 2026	Used as an interactive writing assistant to improve readability and consistency.
ChatGPT (OpenAI)	Assistance with LaTeX formatting and figure/table creation	Figures, tables, appendix, layout formatting	May–June 2026	Used to generate and debug LaTeX code for graphs, tables, and image layouts.
ChatGPT (OpenAI)	Assistance with PDDL and Fast Downward related implementation questions	Experimental setup and implementation	May–June 2026	Used to clarify planner behavior, heuristic configuration, and PDDL modeling issues.
ChatGPT (OpenAI)	Assistance interpreting experimental results and identifying trends	Results and conclusion sections	June 2026	Interpretations were manually verified against the generated data.
ChatGPT (OpenAI)	Suggestions for README structure and GitHub repository organization	Project repository documentation	June 2026	Used for repository presentation and setup instructions.
Perplexity AI	Literature exploration and identification of related sources	Related work and background research	May–June 2026	Used to discover relevant papers and planning-related literature which were independently reviewed before citation.
DeepL Translator	Minor language corrections and translation support	Small text passages	May–June 2026	Used occasionally for wording and language refinement.

TABLE VII: Declaration of AI-assisted tools used during the project.